

Listas

Objetivo da Aula

Ao final desta aula, você será capaz de:

- Entender o que são listas em Python e por que elas existem
- Criar e manipular listas com confiança
- Acessar elementos por índice (positivo e negativo)
- Adicionar e remover itens de forma precisa
- Percorrer listas com loops
- Aplicar listas em cenários reais de testes e automação QA

1. O que são Listas?

Imagine que você precisa testar um sistema de login com **vários usuários diferentes**. Sem listas, você faria assim:

```
# Sem lista – trabalhoso e difícil de escalar
usuario1 = "ana@email.com"
usuario2 = "carlos@email.com"
usuario3 = "maria@email.com"
# E se forem 50 usuários?
```

Com lista, tudo fica organizado em **uma única variável**:

```
# Com lista – organizado e escalável
usuarios = ["ana@email.com", "carlos@email.com", "maria@email.com"]
```

Analogia: Pense em uma lista como uma **planilha de uma coluna**, cada linha tem um valor, e você pode acessar qualquer linha pelo número dela.

Definição:

> Uma lista é uma coleção **ordenada**, **mutável** e que aceita **tipos mistos** de valores.

2. Criando Listas

Listas são criadas com **colchetes** `[]` e os itens são separados por **vírgula**.

```
# Lista de e-mails para teste
emails = ["qa@empresa.com", "admin@empresa.com", "user@empresa.com"]

# Lista de status HTTP que precisamos validar
codigos_http = [200, 201, 400, 401, 404, 500]

# Lista com tipos mistos
dado_misto = ["Ana", 30, True, None]

# Lista vazia (útil para adicionar itens dinamicamente)
resultados = []
```

Características importantes

Característica	Descrição
Ordenada	A ordem dos elementos é mantida sempre
Mutável	Você pode alterar, adicionar e remover itens
Aceita duplicatas	O mesmo valor pode aparecer mais de uma vez
Tipos mistos	Pode misturar strings, números, booleanos, etc.

3. Acessando Elementos por Índice

Cada elemento da lista ocupa uma **posição numerada chamada índice**.

Atenção: Em Python, os índices **começam em 0**, não em 1!

```
ambientes = ["desenvolvimento", "homologação", "produção"]
#           índice 0           índice 1       índice 2

print(ambientes[0]) # desenvolvimento
print(ambientes[1]) # homologação
print(ambientes[2]) # produção
```

Saída:

```
desenvolvimento  
homologação  
produção
```

Visualizando os índices

desenvolvimento	homologação	produção
índice 0	índice 1	índice 2

Dica de QA: Ao acessar listas dinamicamente nos seus testes, sempre valide se o índice existe antes de acessá-lo. Acessar um índice que não existe gera `IndexError` e quebra o teste.

4. Índices Negativos

Python permite acessar elementos **de trás para frente** usando índices negativos. Muito útil quando você quer o **último item** sem saber o tamanho da lista.

```
ambientes = ["desenvolvimento", "homologação", "produção"]  
  
print(ambientes[-1]) # produção      ← último  
print(ambientes[-2]) # homologação   ← penúltimo  
print(ambientes[-3]) # desenvolvimento ← antepenúltimo
```

Mapa completo de índices

desenvolvimento	homologação	produção
índice 0	índice 1	índice 2

índice -3	índice -2	índice -1
-----------	-----------	-----------

Caso prático: pegar o resultado mais recente de uma lista de execuções:

```
execucoes = ["PASSOU", "PASSOU", "FALHOU", "PASSOU"]

ultimo_resultado = execucoes[-1]
print(f"Resultado mais recente:{ultimo_resultado}")
# Resultado mais recente: PASSOU
```

5. Modificando Valores

Listas são **mutáveis** — você pode alterar qualquer elemento diretamente pelo índice.

```
# Cenário: corrigir um e-mail cadastrado errado
usuarios = ["ana@email.com", "carlso@email.com", "maria@email.com"]
#                                     ^ erro de digitação

# Corrigindo o índice 1
usuarios[1] = "carlos@email.com"

print(usuarios)
# ['ana@email.com', 'carlos@email.com', 'maria@email.com']
```

6. Adicionando Elementos

append() — Adicionar no final

```
# Cenário: novos usuários sendo cadastrados durante o teste
usuarios = ["ana@email.com", "carlos@email.com"]

usuarios.append("maria@email.com")
usuarios.append("pedro@email.com")
```

```
print(usuarios)
# ['ana@email.com', 'carlos@email.com', 'maria@email.com',
  'pedro@email.com']
```

`insert()` — Adicionar em posição específica

```
# Cenário: inserir o admin sempre na primeira posição da lista
usuarios = ["ana@email.com", "carlos@email.com"]

usuarios.insert(0, "admin@email.com")

print(usuarios)
# ['admin@email.com', 'ana@email.com', 'carlos@email.com']
```

Comparativo rápido

Método	Onde adiciona	Exemplo
<code>append(valor)</code>	Sempre no final	<code>lista.append("novo")</code>
<code>insert(i, valor)</code>	Na posição i	<code>lista.insert(0, "primeiro")</code>

7. Removendo Elementos

`remove()` — Remover pelo valor

Remove a **primeira ocorrência** do valor informado.

```
# Cenário: remover um usuário que foi desativado
usuarios = ["ana@email.com", "carlos@email.com", "maria@email.com"]

usuarios.remove("carlos@email.com")

print(usuarios)
# ['ana@email.com', 'maria@email.com']
```

Cuidado: Se o valor **não existir** na lista, o `remove()` gera um `ValueError`. Em testes, sempre verifique se o item existe antes de remover.

`pop()` — Remover pelo índice

Remove e **retorna** o elemento na posição informada. Se não passar índice, remove o último.

```
usuarios = ["ana@email.com", "carlos@email.com", "maria@email.com"]

# Remove e captura o item removido
removido = usuarios.pop(1)

print(f"Usuário removido:{removido}")
print(f"Lista atualizada:{usuarios}")
# Usuário removido: carlos@email.com
# Lista atualizada: ['ana@email.com', 'maria@email.com']
```

Comparativo rápido

Método	Remove por	Retorna o item?
<code>remove(valor)</code>	Valor	 Não
<code>pop(índice)</code>	Índice	 Sim
<code>pop()</code>	Último item	 Sim

8. Descobrindo o Tamanho da Lista

Use `len()` para saber quantos elementos existem na lista.

```
# Cenário: validar se o número de resultados retornados pela API está correto
resultados_api = ["item1", "item2", "item3", "item4", "item5"]

total = len(resultados_api)
print(f"Total de resultados:{total}")
# Total de resultados: 5
```

```
# Uso em validação
esperado = 5
if len(resultados_api) == esperado:
    print("Quantidade de resultados correta!")
else:
    print(f"Esperava{esperado}, mas veio{len(resultados_ap
i)}")
```

9. Percorrendo Listas com Loops

Loop simples com **for**

```
# Cenário: imprimir todos os ambientes disponíveis para tes
te
ambientes = ["desenvolvimento", "homologação", "produção"]

for ambiente in ambientes:
    print(f"Testando em:{ambiente}")
```

Saída:

```
Testando em: desenvolvimento
Testando em: homologação
Testando em: produção
```

Loop com índice usando **enumerate()**

Quando você precisa do valor **e** da posição ao mesmo tempo:

```
casos_de_teste = ["Login válido", "Login inválido", "Login
sem senha"]

for indice, caso in enumerate(casos_de_teste):
    print(f"Caso{indice + 1}:{caso}")
```

Saída:

Caso 1: Login válido
Caso 2: Login inválido
Caso 3: Login sem senha

10. Verificando se um Item Existe

Use o operador `in` para checar se um valor está presente na lista.

```
# Cenário: verificar se o status code retornado é um dos esperados
codigos_sucesso = [200, 201, 204]
codigo_recebido = 201

if codigo_recebido in codigos_sucesso:
    print(f"Código{codigo_recebido} é um código de sucesso!")
else:
    print(f"Código{codigo_recebido} não era esperado.")
```

Saída:

Código 201 é um código de sucesso!

Você também pode usar `not in` para verificar a **ausência**:

```
erros_criticos = [500, 502, 503]
codigo = 404

if codigo not in erros_criticos:
    print(f"Código{codigo} não é um erro crítico de servidor.")
```

Resumo dos Métodos

Operação	Sintaxe	Descrição
Criar lista	<code>lista = [a, b, c]</code>	Cria uma nova lista

Operação	Sintaxe	Descrição
Acessar elemento	<code>lista[0]</code>	Acessa pelo índice
Modificar elemento	<code>lista[0] = novo</code>	Altera o valor na posição
Adicionar no final	<code>lista.append(x)</code>	Insere ao final
Adicionar na posição	<code>lista.insert(i, x)</code>	Insere na posição i
Remover por valor	<code>lista.remove(x)</code>	Remove primeira ocorrência
Remover por índice	<code>lista.pop(i)</code>	Remove e retorna o item
Tamanho	<code>len(lista)</code>	Retorna a quantidade de itens
Verificar existência	<code>x in lista</code>	Retorna True ou False
Percorrer	<code>for item in lista</code>	Itera sobre os elementos
Percorrer com índice	<code>enumerate(lista)</code>	Retorna índice + valor